

Pytest Framework – Software Testing Complete Notes

Exhaustive Study Notes covering Discovery, Assertions, Fixtures, Scopes, Parametrization, Markers & Reporting

1. Pytest Overview & Introduction

Pytest is a mature, full-featured Python testing framework that makes it easy to write small, readable tests, and scales to support complex functional testing for applications and libraries.

Why Pytest over unittest? • No Boilerplate: Does not require creating classes inheriting from `unittest.TestCase`. • Plain assert statements: Uses standard Python `assert` expression instead of `self.assertEqual()`. • Powerful Fixture System: Modular dependency injection with automatic setup/teardown. • Rich Ecosystem: Support for 300+ plugins (e.g. `pytest-xdist`, `pytest-html`, `pytest-cov`).

```
# Simple Pytest Test Function
def add(a, b):
    return a + b

def test_add_success():
    assert add(2, 3) == 5
```

■ **Key Takeaway:** Pytest automatically rewrites Python assert statements to provide detailed failure context and variable values.

2. Test Naming Conventions & Discovery

Pytest automatically discovers and executes tests following strict naming conventions: • Test Files: Files named `test_*.py` or `*_test.py`. • Test Functions: Functions named `test_*`. • Test Classes: Classes named `Test*` (without `__init__` constructor) containing `test_*` methods.

CLI Execution Commands: • `pytest`: Run all discovered tests. • `pytest -v`: Verbose mode (shows test names and PASS/FAIL status). • `pytest -s`: Disable output capture (prints `print()` statements to console). • `pytest -k "login"`: Run tests matching string expression 'login'. • `pytest test_auth.py`: Run tests in a specific file.

Pytest Test Discovery & Execution Workflow Pipeline



Diagram: Pytest Test Discovery & Execution Workflow Pipeline.

3. Assertions & Exception Testing

Pytest utilizes standard Python `assert` expressions. When an assertion fails, Pytest performs assertion introspection to display intermediate expression values.

Testing Expected Exceptions: Use `pytest.raises(ExceptionType)` context manager to verify that code raises a specific exception.

```
import pytest

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

def test_divide_by_zero_exception():
    with pytest.raises(ValueError) as exc_info:
        divide(10, 0)
    assert "Cannot divide by zero" in str(exc_info.value)
```

4. Pytest Fixtures & conftest.py

Fixtures feed preconditions, test data, database state, or browser instances to test functions via argument dependency injection.

Fixture Scopes: • ``function`` (default): Executed before and after each test function. • ``class``: Executed once per test class. • ``module``: Executed once per test script file. • ``session``: Executed once for the entire test suite run.

Setup & Teardown with ``yield``: Code before ``yield`` acts as Setup; code after ``yield`` acts as Teardown.

`conftest.py` File: Special configuration file placed in test root directory. Fixtures defined in ``conftest.py`` are automatically accessible to all test files without requiring explicit imports.

```
# conftest.py
import pytest
from selenium import webdriver

@pytest.fixture(scope="function")
def driver():
    # Setup Phase
    browser = webdriver.Chrome()
    browser.maximize_window()
    yield browser
    # Teardown Phase
    browser.quit()
```

Pytest Fixture Scope Hierarchy & Lifecycle (yield)

session Scope (Runs once per test suite)
Database Connection / Global Server Launch

module Scope (Runs once per test file)
Test Data Pre-loading / API Token Auth

class Scope (Runs once per Test Class)
Shared Page Objects / Test Context

function Scope (Runs before/after each test function)
Browser Launch -> yield -> Browser quit()

Diagram: Pytest Fixture Scope Hierarchy & Lifecycle (yield).

5. Parametrization & Data-Driven Testing

The ``@pytest.mark.parametrize`` decorator enables running the same test function across multiple datasets without writing repetitive code loops.

```
import pytest

@pytest.mark.parametrize("email, password, expected", [
    ("valid@example.com", "Pass123!", True),
    ("invalid-email", "Pass123!", False),
    ("valid@example.com", "wrongpass", False),
])
def test_login_validation(email, password, expected):
    result = perform_login(email, password)
    assert result == expected
```

6. Pytest Markers & pytest.ini Config

Markers categorize, skip, or modify test execution behavior.

Built-in Markers: • `@pytest.mark.skip(reason="...")`: Unconditionally skips test. • `@pytest.mark.skipif(condition, reason="...")`: Conditionally skips test. • `@pytest.mark.xfail(reason="...")`: Marks test as expected failure.

Custom Markers & pytest.ini: Register custom markers inside `pytest.ini` configuration file.

```
# pytest.ini configuration file
[pytest]
minversion = 7.0
addopts = -v --tb=short
markers =
    smoke: High priority smoke tests
    regression: Full regression test suite
    api: API integration tests
```

7. Parallel Execution & HTML Reporting Plugins

Pytest extensibility via PyPI plugins: • `pytest-xdist`: Runs tests in parallel across CPU cores (`pytest -n 4`). • `pytest-html`: Generates standalone visual HTML test reports (`pytest --html=report.html --self-contained-html`). • `pytest-cov`: Measures statement code coverage (`pytest --cov=myapp --cov-report=html`).

■ **Key Takeaway:** Parallel execution using `pytest-xdist` reduces test suite run times by up to 75% on multi-core CI/CD runners.

Pytest Automation Testing Ecosystem & Plugin Architecture

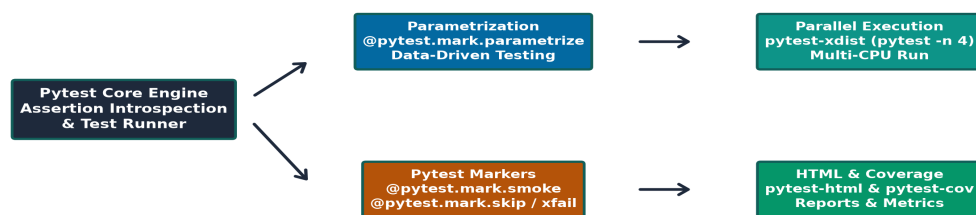


Diagram: Pytest Automation Testing Ecosystem & Plugin Architecture.

8. Pytest Quick Reference & Viva Questions

Top Interview Questions & Key Answers: 1. Q: What is `conftest.py` in Pytest? A: It is a local plugin file used to define shared fixtures and hooks across tests without needing imports. 2. Q: How does Pytest handle fixture teardown? A: Code written after the `'yield'` statement in a fixture function executes during teardown. 3. Q: How do you run tests in parallel using Pytest? A: Using the `'pytest-xdist'` plugin with command `'pytest -n auto'` or `'pytest -n 4'`.